

Лабораторная работа 3

ОРГАНИЗАЦИЯ СВЯЗИ MASTER-DETAIL МЕЖДУ ДАННЫМИ

Цель работы: ознакомиться с механизмом связи Master–Detail и особенностями редактирования данных с такой связью; изучить способы выборки данных на основе изменения SQL-кода во время работы приложения; разработать клиентское приложение баз данных, позволяющее работать со связанными данными по принципу Master–Detail

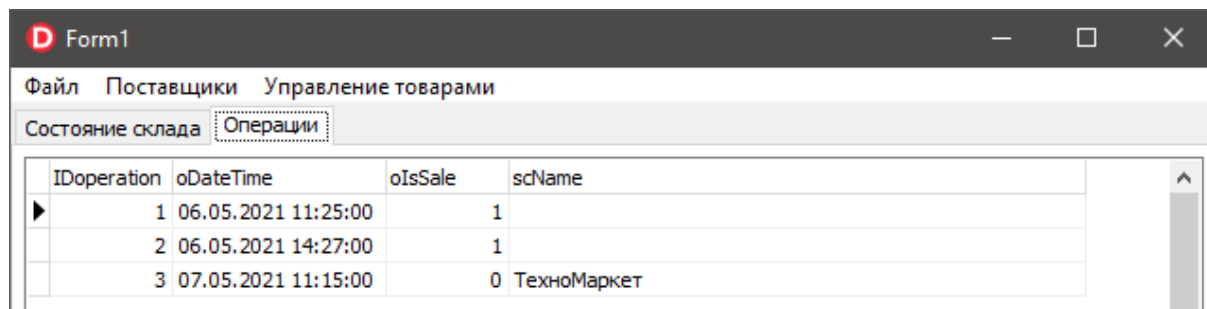
ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с теоретическими сведениями.
2. Открыть копию проекта лабораторной работы №2.
3. Создать необходимые представления.
4. Добавить к проекту компоненты для работы с новыми представлениями.
5. Настроить между данными связь **Master–Detail**.
6. Организовать выбор типа операции при помощи **RadioGroup**.
7. Реализовать редактирование списка товаров определенной операции.
8. Ответить на контрольные вопросы.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Master–Detail представление связанных данных при котором основные данные отображаются в виде списка, а их подробное описание или связанные данные только при выборе конкретного значения из этого списка. Например, так обычно отображается информация в почтовых клиентах.

Для реализации текущего задания необходимо использовать копию проекта, сформированного в предыдущей лабораторной работе. В текущей версии программы отображается представление об операциях с товарами (рис. 3.1). Однако приложение не показывает, на какую сумму выполнены операции и не ясно какие товары участвовали в операциях.



IDoperation	oDateTime	oIsSale	scName
1	06.05.2021 11:25:00	1	
2	06.05.2021 14:27:00	1	
3	07.05.2021 11:15:00	0	ТехноМаркет

Рис. 3.1. Вывод сведений об операциях в приложении

Для реализации функциональных особенностей приложения нам понадобится расширить набор представлений базы данных и модифицировать наборы данных приложения.

3.1 СОЗДАНИЕ ПРЕДСТАВЛЕНИЙ И МОДИФИКАЦИЯ НД

Сначала сформируем представление **view_operation_goods_list**, которое покажет, какие товары (с учетом типа) и в каком количестве участвовали в операциях. При этом, также будем отображать цену товара. Для этого необходимо сгенерировать *sql*-запрос. Будем использовать конструктор запросов. Перенесем в область конструктора таблицы **op_goods_list**, **goods_catalog** и **type_goods** и выберем поля как показано на рис. 3.2.

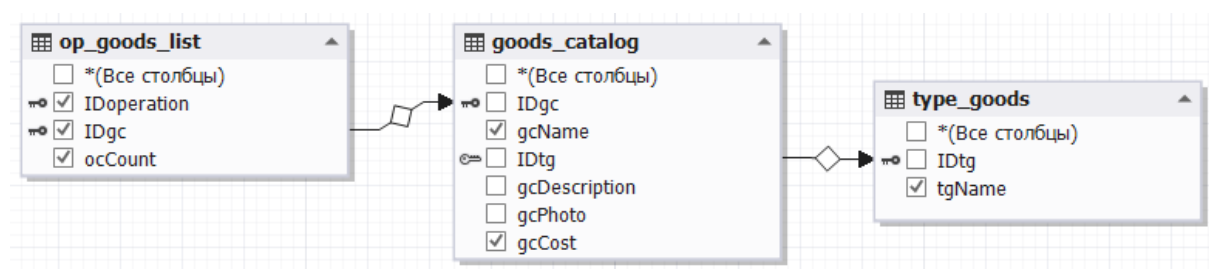


Рис. 3.2. Конструирование запроса для нового представления

Используя за основу полученный текст SQL запроса, добавим в него строчку определяющую поле **ogcFCost** как произведение полей **ocCount** и **gcCost**. Сформированный код сохраним как представление (рис. 3.3). Пример выводимых данных приведен на рис. 3.4.

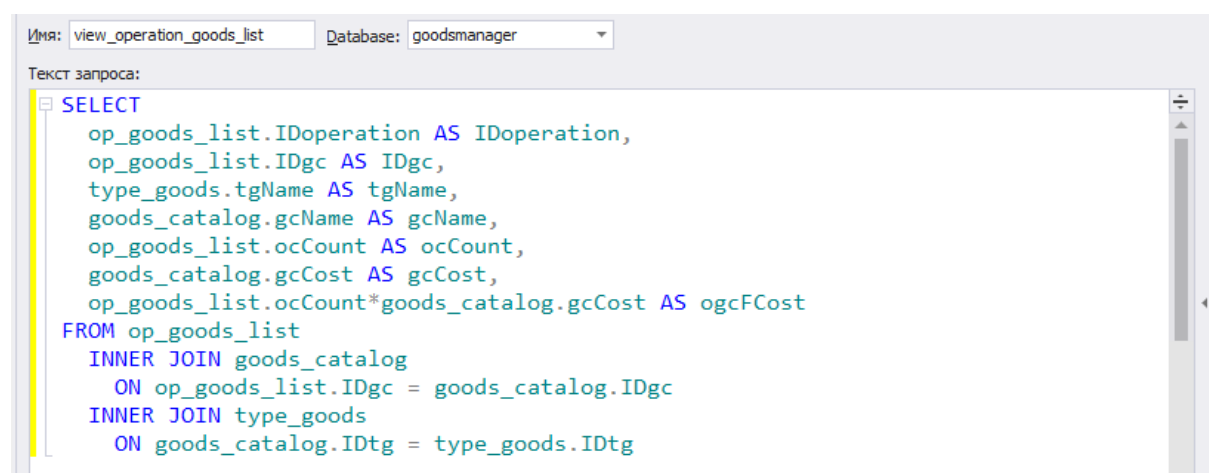


Рис. 3.3. SQL-запрос нового представления

IDoperation INT(11)	IDgc INT(11)	tgName VARCHAR(50)	gcName VARCHAR(100)	ocCount INT(11)	gcCost DECIMAL(10, 2)	ogcFCost DECIMAL(20, 2)
1	1	Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	1	25499,00	25499,00
2	5	Системные блоки	ASUS ROG Strix GL10CS-RU077T	1	64990,00	64990,00
2	6	Мониторы	Samsung F27T450FQI	2	15199,00	30398,00
3	2	Ноутбуки	Lenovo IdeaPad 3 15IGL05 серый	5	27955,00	139775,00
3	3	Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	2	49999,00	99998,00
3	4	Системные блоки	DEXP Mars E317	4	41999,00	167996,00

Рис. 3.4. Выводимые данные представлением *view_operation_goods_list*

Теперь необходимо отобразить полную стоимость товаров, участвовавших в конкретной операции. Для этого необходимо сделать группировку данных с помощью параметра **GROUP BY** оператора **SELECT**. Параметр **GROUP BY** используется для определения групп выходных строк, к которым могут применяться агрегатные функции (**COUNT**, **MIN**, **MAX**, **AVG**, **SUM** и др.). Если при наличии параметра **GROUP BY**, в операторе **SELECT** отсутствуют агрегатные функции, то запрос просто вернет по одной строке из каждой группы.

Поскольку представление **view_operation_goods_list** содержит все необходимые данные то мы будем использовать его для построения запроса отображающего полную стоимость товаров, участвующих в операции. *SQL*-код запроса будет выглядеть следующим образом:

```
SELECT
  IDoperation AS IDoperation,
  SUM(ogcFCost) AS oFullCost
FROM view_operation_goods_list
GROUP BY IDoperation
```

Сохраним код данного запроса как представление с именем **view_operations_fullcost**. Результат выполнения представлен на рис. 3.5:

IDoperation INT(11)	FullCost DECIMAL(42, 2)
1	25499,00
2	103788,00
3	407769,00

Рис. 3.5. Общая стоимость товаров по определенным операциям.

Теперь изменим представление **view_goods_operations**. Для этого войдем в режим дизайнера и добавим туда созданное представление (рис. 3.6). При помощи зажатой левой кнопки мыши соединим поля **IDoperation** из таблицы **operations** с представлением **view_operations_fullcost**, а также отметим поле **oFullCost** для вывода. В настройки связи необходимо указать **Выбрать все строки из Operations**.

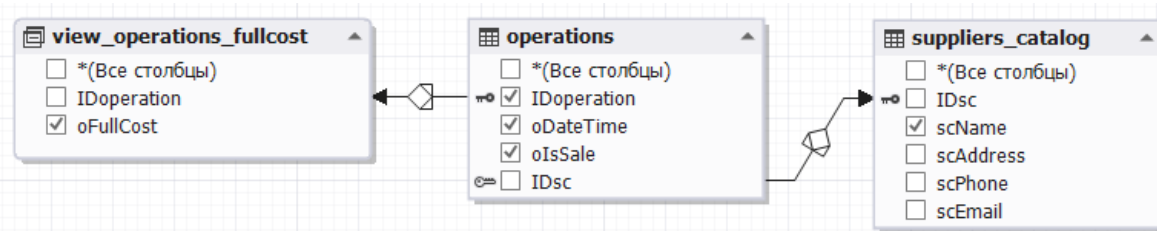


Рис. 3.6. Модификация представления *view_goods_operations*

В нашем примере тип операции отображается в числовом формате. Для пользователя программы такая форма отображения не очень удобна, в связи с этим воспользуемся возможностью изменения вывода в SQL запросе. Для условного отображения текста воспользуемся конструкцией с применением функции **IF()** в операторе **SELECT**. Функция имеет следующую форму записи:

IF(условие, значение1, значение2)

В функция **IF()** если условие равно значению **ИСТИНА**, то она возвращает **значение1**, в противном случае – **значение2**. В зависимости от контекста функция **IF()** может возвращать либо числовое, либо строковое значение. Это определяется формой записи значений. Для вывода текстовых значений нужно использовать одинарные кавычки, например:

SELECT IF(1<2, 'да', 'нет');

В случае вывода числовых значений достаточно их просто указать:

SELECT IF(1>2, 2, 3);

Добавим в блок выборки данных конструкцию (рис. 3.7):

IF(operations.oIsSale = 1, 'Продажа', 'Поставка') AS opType,

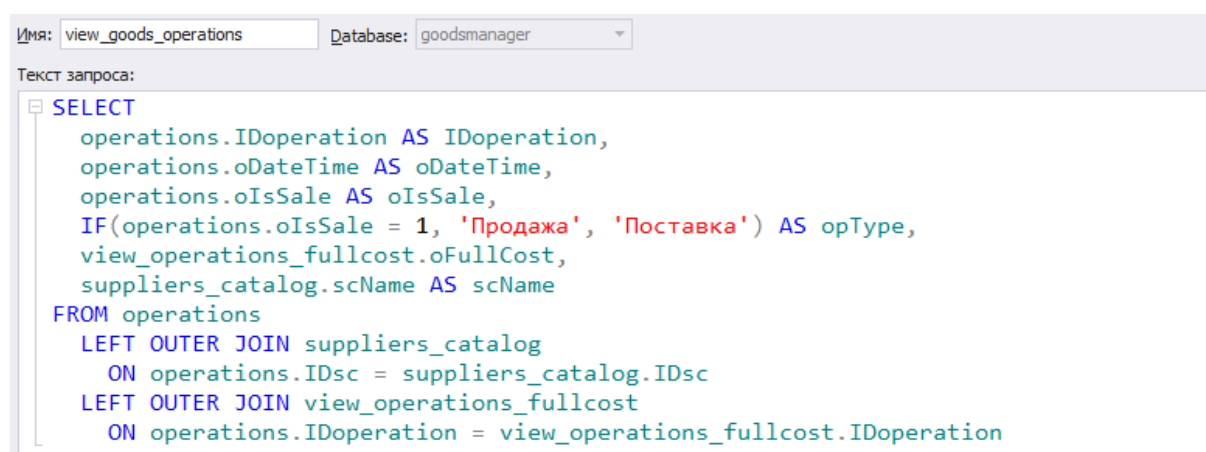


Рис. 3.7. Текст запроса представления *view_goods_operations*

Теперь в выводимых данных представления *view_goods_operations* типа операции будет отображаться в виде текстовой записи, а также будет отражена стоимость товаров в этой операции (рис 3.8).

IDoperation INT(11)	oDateTime DATETIME	oIsSale TINYINT(1)	opType VARCHAR(8)	oFullCost DECIMAL(42, 2)	sdName VARCHAR(50)
1	06.05.2021 11:25:00	1	Продажа	25499,00	(null)
2	06.05.2021 14:27:00	1	Продажа	95388,00	(null)
3	07.05.2021 11:15:00	0	Поставка	407769,00	ТехноМаркет

Рис. 3.8. Данные *view_goods_operations*

Создадим еще одно представление, которое в будущем позволит компактно отображать информацию о товаре и его цене, например, в выпадающих списках (компонент **TDBLookupComboBox** позволяет отображать содержимое только одного поля).

Для этого необходимо использовать единое поле, которое будет содержать данные полей **gcName** и **gcCost** из таблицы **goods_catalog**. При этом, для контроля значений в разрабатываемом приложении в будущем нам понадобятся сведения о количестве товара доступного на складе.

С учетом этого создадим новое представление **view_selector_goods**, которое будет настроено следующим образом (Рис 3.9).

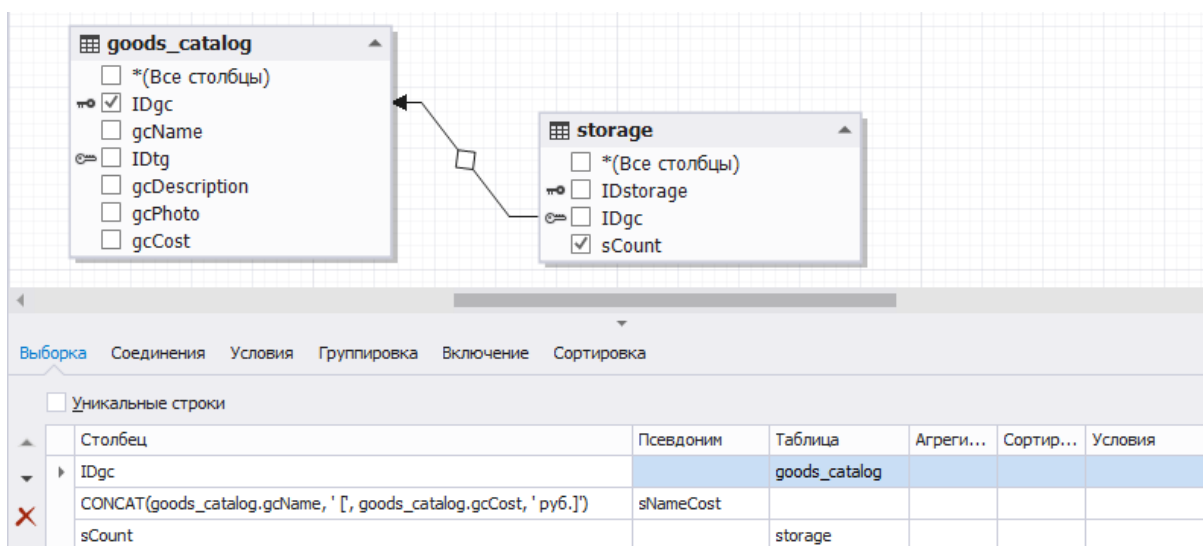


Рис. 3.9. Настройка представления *view_selector_goods*

Применяемая в запросе функция **CONCAT** предназначена для сложения строк при выборке из базы данных. Строками обычно выступают поля таблиц, но могут быть использованы и строковые константы. В общем виде синтаксис с использованием функции **CONCAT** выглядит следующим образом:

```
SELECT
    CONCAT(поле1, поле2...) AS псевдоним
FROM имя_таблицы
```

Как альтернатива может быть использована функция **CONCAT_WS**, которая также предназначена для сложения строк при выборке из базы данных, но уже с использованием разделителя. Синтаксис использования данной функции имеет следующую форму:

```
SELECT
    CONCAT_WS(разделитель, поле1, поле2...) AS псевдоним
FROM имя_таблицы
```

По результатам настройки для представления будет сформирован код, который показан на рисунке 3.10

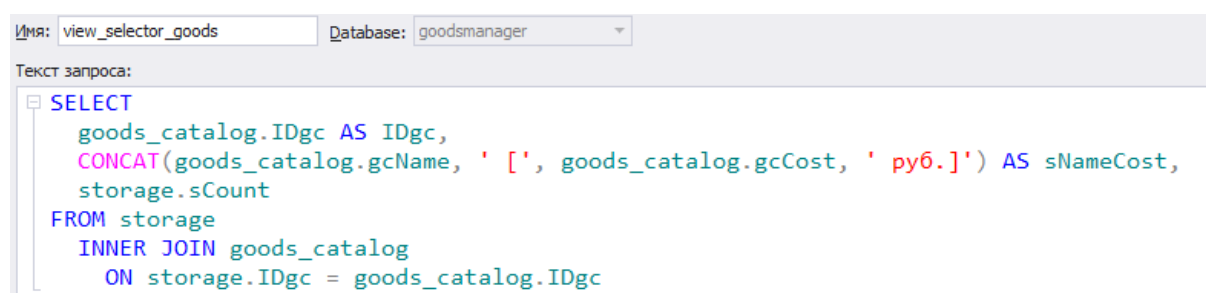


Рис. 3.10. Сформированный SQL-код представления *view_selector_goods*

После сохранения, представление *view_selector_goods* будет выводить следующие данные, показанные на рис. 3.11.

IDgc INT(11)	sNameCost VARCHAR(120)	sCount INT(11)
1	Acer Aspire 1 A115-32-C2Z1 серебристый [25499.00 руб.]	5
2	Lenovo IdeaPad 3 15IGL05 серый [27955.00 руб.]	7
3	ASUS VivoBook M513UA-BQ080 черный [49999.00 руб.]	6
4	DEXP Mars E317 [41999.00 руб.]	4
5	ASUS ROG Strix GL10CS-RU077T [64990.00 руб.]	5

Рис. 3.11. Фрагмент данных *view_selector_goods*

Для вновь созданных представлений в нашем программном проекте на **DataModule2** необходимо добавить несколько компонентов **TADOQuery** и **TDataSource**. Настройка будет происходить аналогично тому, как это было сделано во второй лабораторной работе.

В связи с тем, что были внесены изменения в представление **view_goods_operations** то в НД **Q_v_goods_operations** необходимо обновить набор полей при помощи редактора полей (*Fields Editor*), а также задать обработчик события **AfterScroll**. Реализация кода обработчика будет аналогична тому, как у **Q_v_goods_catalog** в предыдущей лабораторной работе, при этом в методе **Locate** для синхронизации данных будет использовано поле **IDoperation**.

Для представления **view_operation_goods_list** будем использовать компонент **TADOQuery**, который назовем **Q_v_operation_goods_list**.

У него аналогичным образом пропишем обработчик событий **AfterScroll**, но уже с учетом использования составного первичного ключа:

```
T_op_goods_list.Locate('IDoperation;IDgc',  
    VarArrayOf([Q_v_operation_goods_listIDoperation.Value,  
        Q_v_operation_goods_listIDgc.Value]), [])
```

Следует обратить внимание, что в методе **Locate** используется сразу два поля **IDoperation** и **IDgc**, поскольку имеется составной первичный ключ. Функция **VarArrayOf** создает одномерный массив значений **Variant** (могут храниться данные почти любых типов данных). Для её использования необходимо чтобы в блоке **uses** в самом верху программного кода модуля был прописан заголовочный файл **System.Variants** (для старых версий **RAD Studio** просто **Variants**).

Теперь перейдем к изменению внешнего вида главной формы (**Form1**). Добавим на вкладку с составом операций еще один компонент **TDBGrid**, и свяжем его с **Q_v_operation_goods_list**. Далее настроим визуальное отображение. Для компонента **DBGrid3** установим свойство **Align** в значение **alBottom**. Для того чтобы компоненты визуально не сливались, установим им свойство **AlignWithMargins** в значение **True**. Настроим отображение столбцов как показано на (рис. 3.12).

Далее можно разместить компонент **TLabel** который опишет, какие данные отображаются в **DBGrid3**. Зададим ему свойство **Align** в значение **alBottom**, а свойству **Alignment** (отвечает за выравнивание текста) назначим значение **taCenter**.

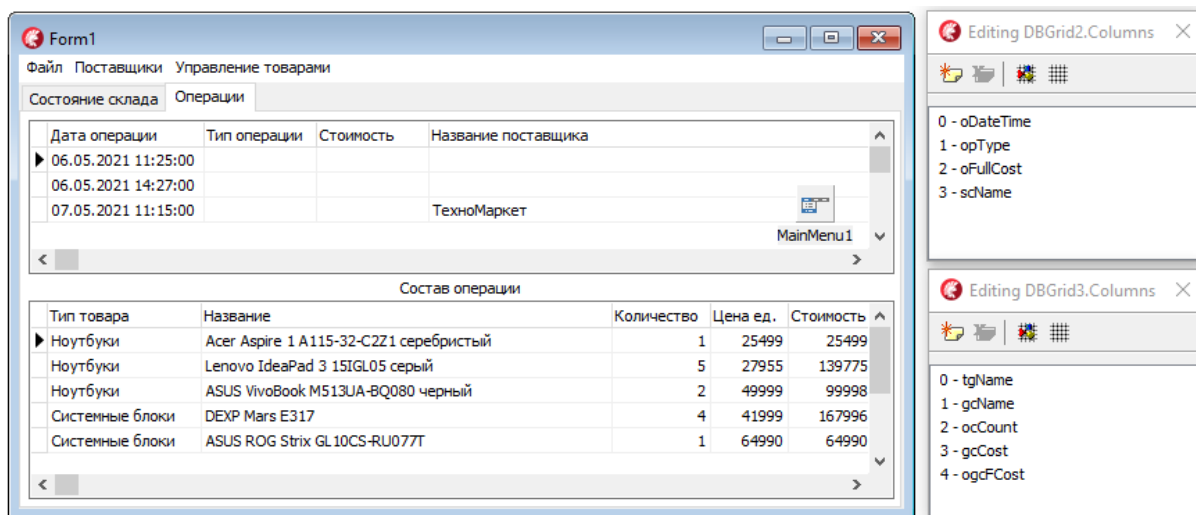


Рис. 3.12. *Form1* после настройки компонентов на вкладке **Операции**, редакторы столбцов компонентов **DBGrid2** и **DBGrid3**

После организации интерфейса, перейдем к непосредственной настройке отображения данных с использованием механизма Master–Detail.

3.2 НАСТРОЙКА ОТОБРАЖЕНИЯ ДАННЫХ С ИСПОЛЬЗОВАНИЕМ МЕХАНИЗМА MASTER–DETAIL

Таблицы **operations** и **op_goods_list** находятся в отношении «один-ко-многим». В таком случае для отображения данных часто применяется механизм **Master–Detail**, в нашем случае при выборе концертной операции из списка будем отображать данные о составе товаров, участвующих в ней.

Связь **Master–Detail** для компонентов **TADOTable** настраивается визуально. Настроим связь между таблицами **operations** и **op_goods_list**. Для этого у компонента **T_op_goods_list** установим свойство **MasterSource** в значение **DS_T_operations**. Далее перейдем на свойство **MasterFields** и нажмем кнопку с многоточием. В появившемся окне **Field Link Designer** установим параметры связи (рис. 3.13), выбрав в обоих списках поле **IDoperation** и нажав кнопку **Add**.

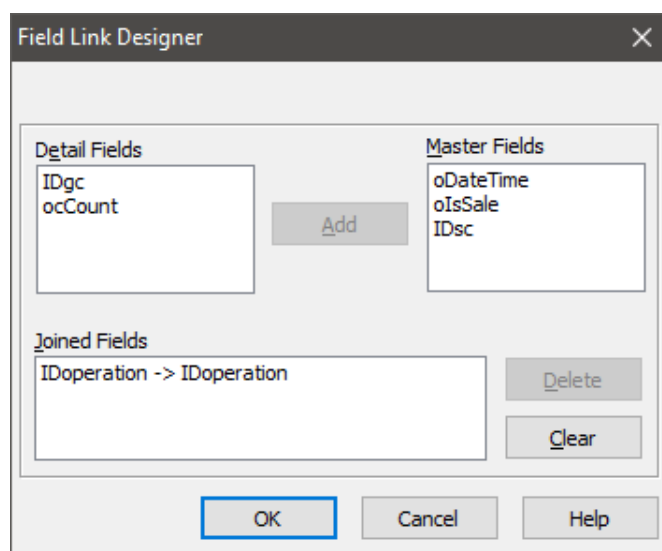


Рис. 3.13. Окно настройки соединения полей **Field Link Designer**

Поскольку для отображения информации о составе операций используется **Q_v_operation_goods_list**, то также настроим для него связь **Master–Detail** с **Q_v_goods_operations**.

Для этого нам необходимо изменить *SQL* код прописанный в компоненте **Q_v_operation_goods_list** на следующий:

```
SELECT *  
FROM view_operation_goods_list  
WHERE view_operation_goods_list.IDoperation = :IDoperation
```

Как можно заметить, в *SQL*-запросе появился блок *WHERE* в котором прописано условие, выполняющее проверку равенства. В нем **:IDoperation** означает параметр, значение которого берется из набора данных посредством источника указанного в свойстве **DataSource** или же задается пользователем (в настройках или программном коде).

В нашем случае значение параметра будет получено из набора данных **Q_v_goods_operations**. Для этого у **Q_v_operation_goods_list** свойству **DataSource** установим значение **DS_Q_v_goods_operations**, затем свойству **Active** – значение **True**. Теперь при выборе из списка какой-либо операции, будут отображаться принадлежащие ей товары (рис. 3.14).

Дата операции	Тип операции	Стоимость	Название поставщика
06.05.2021 11:25:00	Продажа	25499	
06.05.2021 14:27:00	Продажа	95388	
07.05.2021 11:15:00	Поставка	407769	ТехноМаркет

Тип товара	Название	Количество	Цена ед.	Стоимость
Ноутбуки	Lenovo IdeaPad 3 15IGL05 серый	5	27955	139775
Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	2	49999	99998
Системные блоки	DEXP Mars E317	4	41999	167996

Рис. 3.14. Демонстрация работы *Master–Detail*, для отображения состава операции

При наличии большого количества данных (в нашем случае сведений о продажах и поставках) целесообразно применять выборку на основе модификации SQL-запроса. Рассмотрим, как это сделать.

3.3 НАСТРОЙКА ВЫБОРКИ ДАННЫХ ПРИ ПОМОЩИ МОДИФИКАЦИИ SQL

При наличии большого количества данных (в нашем случае сведений о продажах и поставках) целесообразно применять выборку, которая будет давать возможность просмотреть все операции, либо только продажи или поставки.

Для настройки элементов визуального выбора разместим на вкладке **Операции** компонент **TPanel** и зададим свойству **Align** значение **alTop**. Настройку внешнего вида панели можно осуществить при помощи свойств **BevelInner**, **BevelKind** и **BevelOuter**. Для того чтобы панель визуально слилась с формой необходимо задать значение **bvNone** для этих свойств.

Управление видом отображаемых данных будет происходить при помощи компонента **TRadioGroup**. Разместим его на панель и назначим свойству **Align** значение **alLeft**. Нажав на кнопку с многоточием у свойства **Items** и в появившемся редакторе в каждой строке пропишем значе-

ния: **Все операции**, **Продажи**, **Поставки**. Нажав кнопку **Ok**, увидим, как отображаются элементы. Для того чтобы элементы были расположены горизонтально необходимо задать значение **3** свойству **Columns** и для выбора элемента по умолчанию установить свойству **ItemIndex** значение **0** (выбор **Все операции**). Также настроим свойство **Caption** – пропишем **Выбор операций** (рис. 3.15).

Рис. 3.15. Результат настройки компонента *RadioGroup*

Для организации выбора отображаемых операций на событие **OnClick** у компонента **RadioGroup1** пропишем следующий код:

```
procedure TForm1.RadioGroup1Click(Sender: TObject);
begin
    // Отключаем связь с визуальным компонентом
    DataModule2.Q_v_goods_operations.DisableControls;
    // Деактивируем НД Q_v_goods_operations
    DataModule2.Q_v_goods_operations.active := false;
    // Очищаем SQL код в НД Q_v_goods_operations
    DataModule2.Q_v_goods_operations.SQL.Clear;
    // Добавляем базовую часть SQL запроса на выбор данных
    DataModule2.Q_v_goods_operations.SQL.Add
        ('SELECT * FROM view_goods_operations ');
    // В зависимости от выбранного элемента в RadioGroup1 добавляем
    // условие выборки в SQL запрос
    if RadioGroup1.ItemIndex = 1 then
        DataModule2.Q_v_goods_operations.SQL.Add('WHERE oIsSale = 1 ');
    if RadioGroup1.ItemIndex = 2 then
        DataModule2.Q_v_goods_operations.SQL.Add('WHERE oIsSale = 0 ');
    // Активируем НД Q_v_goods_operations
```

```

DataModule2.Q_v_goods_operations.active := true;
// Включаем связь с визуальным компонентом
DataModule2.Q_v_goods_operations.EnableControls;
end;

```

Теперь можно скомпилировать приложение и проверить работоспособность выбора операций через настроенный компонент **TRadioGroup**.

3.4 РЕДАКТИРОВАНИЕ ДАННЫХ С УЧЕТОМ МЕХАНИЗМА MASTER–DETAIL

На форме редактирования состава операций будем отображать полную стоимость товаров в операции (например, как итог в чеке). Для этого необходимо в **Q_v_operations_fullcost** прописать следующий запрос:

```

SELECT *
FROM view_operations_fullcost
WHERE view_operations_fullcost.IDoperation = :IDoperation

```

Поскольку в списке товаров, которые можно добавить в текущую операцию должны отображаться только отсутствующие в ней товары, то для компонента **Q_v_selector_goods** сделаем следующий *SQL*-запрос:

```

SELECT IDgc, sNameCost, sCount
FROM view_selector_goods
WHERE view_selector_goods.IDgc NOT IN
    (SELECT IDgc FROM view_operation_goods_list
     WHERE view_operation_goods_list.IDoperation = :IDoperation)
or view_selector_goods.IDgc = :IDgcSelected

```

Далее для **Q_v_operations_fullcost** и **Q_v_selector_goods** свойству **DataSource** зададим значение **DS_T_operations**. После чего активируем компоненты, установив свойство **Active** в значение **True**.

Параметр **:IDgcSelected** необходим для передачи в *SQL*-запрос идентификатора выбранного пользователем товара, и в случае если он не был создан автоматически, его необходимо создать через свойство **Parameters** компонента **Q_v_selector_goods** (рис. 3.16).

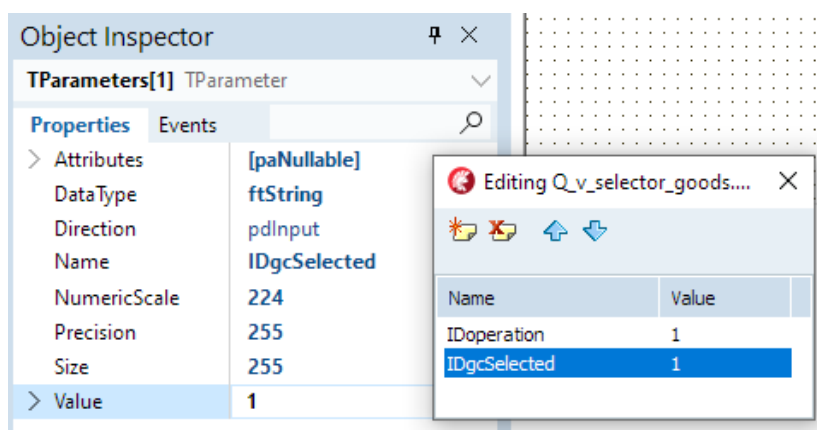


Рис. 3.16. Добавление параметров к *SQL*-запросу

Для добавления поставок или оформления продаж товаров будем использовать новую форму (добавьте её через меню **File**, затем **New** и выберите **VCL Form – Delphi**). Поскольку данные об операциях с товарами хранятся в одной таблице, то на новой форме (**Form5**) предусмотрим одновременное добавление записей продаж и поставок. Различие будет заключаться в скрытии/отображении блока с указанием поставщика.

Для начала поместим панель, которая будет отвечать за настройку параметров самой операции (назовём её **P_EditOperationInfo**). На панели разместим два компонента **TGroupBox**, которые будут отвечать за настройку параметров даты/времени операции и редактирование поставщика. Назовем компоненты **GB_DateTime** и **GB_Supplier** соответственно. В компонент **GB_DateTime** поместим компоненты **TDatePicker** и **TTimePicker**. Для свойства **TimeFormat** компонента **TimePicker1** зададим значение **hh:mm:ss**.

Далее в области компонента **GB_Supplier** поместим компонент **TDBLookupComboBox**, который настроим следующим образом:

Таблица 3.1

Свойства компонента **TDBLookupComboBox** для редактирования поставщика

Свойство	Значение
Name	DBL_supplier
DataSource	DataModule2.DS_T_operations
DataField	IDsc
ListSource	DataModule2.DS_T_supplier_catalog
ListField	scName
KeyField	IDsc

В событии **OnChange** компонента **DatePicker1** пропишем обработчик внесения изменения в поле **oDateTime**:

```
procedure TForm5.DatePicker1Change(Sender: TObject);
begin
    // Если во время активности формы обнаруживаем изменение времени
    // относительно хранящегося в БД
    if Form5.Active and (DataModule2.T_operations.oDateTime.Value <>
        (DatePicker1.Date + TimePicker1.Time)) then
    begin
        // Переводим НД в режим редактирования если он не находится
        // в данном режиме
        if DataModule2.T_operations.State = dsBrowse then
            DataModule2.T_operations.edit;
        // Формируем тип TDateTime (выражается как сумма TDate и TTime)
        DataModule2.T_operations.oDateTime.Value := DatePicker1.Date +
            TimePicker1.Time;
    end;
end;
```

Поскольку в модуле **Unit5** нет информации о библиотеке по работе с базами данных, то необходимо в самом начале файла в блоке **Uses** прописать **Data.DB** (для старых версий Delphi достаточно прописать **DB**).

Теперь **точно такой же код** пропишем для события **OnChange** компонента **TimePicker1**.

Теперь на панель **P_EditOperationInfo** поставим кнопку **Изменить данные** и дадим ей имя **B_ModifyOperationInfo**. В обработчике **OnClick** кнопки пропишите следующий код:

```
procedure TForm5.B_ModifyOperationInfoClick(Sender: TObject);
begin
    // Если набора данных в режиме редактирования
    if DataModule2.T_operations.State = dsEdit then
        // Запоминаем изменения
        DataModule2.T_operations.post;
end;
```

После настройки элементов форма будет выглядеть следующим образом (рис 3.17)

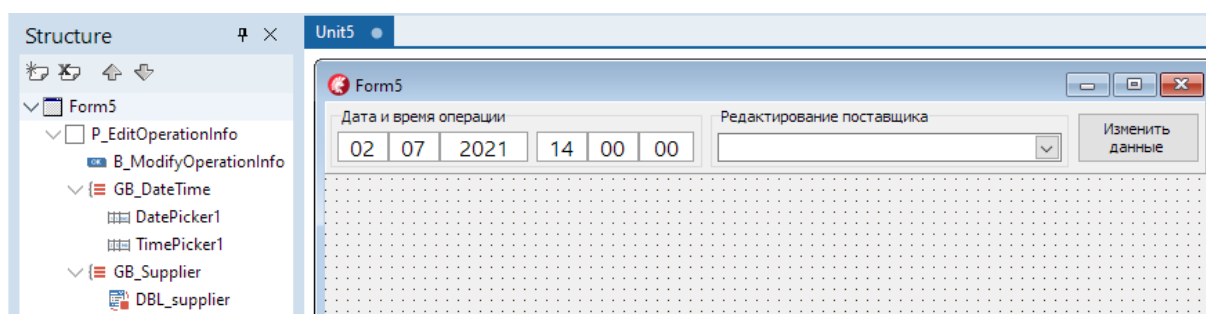


Рис. 3.17. Вид формы после настройки компонентов редактирования данных операции

Под панелью **P_EditOperationInfo** расположим **TDBGrid** (рис. 3.18), который свяжем с **DS_Q_v_operation_goods_list**. В редакторе столбцов настроим поля как у **DBGrid3** на **Form1**, а также настроим выделение целой строки, переведя свойство **options (dgRowSelect)** в значение **True**.

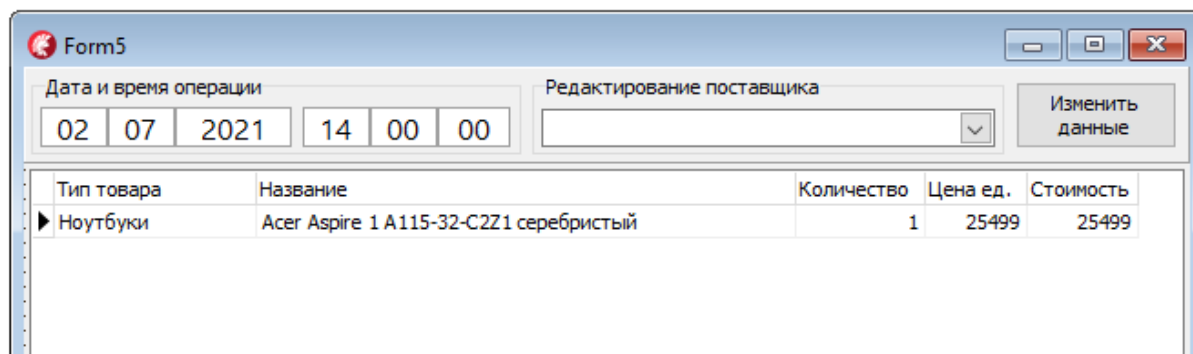


Рис. 3.18. Добавление *DBGrid* для отображения товаров

Для выполнения редактирования данных таблицы **op_goods_list** разместим на форме две панели (дадим им имена **P_BaseControl** и **P_ModifyData**). Для настройки расположенияотрегулируем у компонентов параметры выравнивания (свойство **Align**). У панели **P_ModifyData** свойство **Visible** переведем в значение **False** (рис. 3.19).

Редактирование таблицы **op_goods_list** будет осуществляться по аналогии с тем, как это было в случае с редактированием каталога товаров. Разместим на панели кнопки для вызова основных действий (рис. 3.19 – 1).

Дополнительно на панели **P_BaseControl** поместим отдельную кнопку **Завершить оформление (B_CompleteModifyOperation)** (рис. 3.19 – 2), которая будет отвечать за завершение операции (закрытии формы), а также компонент **TDBText** который будет отвечать за вывод полной стоимости товаров, участвующих в операции (рис. 3.19 – 3).

Настроим компонент **DBText1** задав свойству **DataSource** значение **DataModule2.DS_Q_v_operations_fullcost**, а свойству **DataField** значение **oFullCost**.

Пропишем код обработчика события **OnClick** кнопки **B_CompleteModifyOperation**.

```
procedure TForm5.B_CompleteModifyOperationClick(Sender: TObject);
begin
    // Закрываем форму
    Form5.Close;
end;
```

Рис. 3.19. *Form5* с необходимыми компонентами

Для изменения значения полей в НД **T_op_goods_list** нам понадобится один компонент **TDBLookupComboBox** (рис. 3.19 – 4) для выбора

товара и один компонент **TDBEdit** (рис. 3.19 – 5) для задания количества товаров в поле **ocCount** (разместим их на панель **P_ModifyData**).

Таблица 3.2

Свойства компонента **TDBLookupComboBox** для выбора товара

Свойство	Значение
Name	DBL_goods_selector
DataSource	DataModule2.DS_T_op_goods_list
DataField	IDgc
ListSource	DataModule2.DS_Q_v_selector_goods
ListField	sNameCost
KeyField	IDgc

Теперь разместим компоненты **TLabel** и настроим у них свойство **Caption**. После настройки компонентов форма примет завершённый вид. Далее необходимо прописать все обработчики событий.

Обработчик **OnClick** кнопки **Добавить товар** (имя **B_AppendGoods**):

```
procedure TForm5.B_AppendGoodsClick(Sender: TObject);
begin
  if DataModule2.T_op_goods_list.State = dsBrowse then
  begin
    DBGrid1.Enabled := false; // Деактивируем компонент DBGrid1
    // Указываем в параметре, что товар не выбран
    DataModule2.Q_v_selector_goods.Parameters[1].Value := -1;
    // Обновляем НД списка товаров для DBL_goods_selector
    DataModule2.Q_v_selector_goods.Requery();
    // Включаем режим добавления
    DataModule2.T_op_goods_list.Append;
    P_ModifyData.Visible := true; // Показываем панель
    P_BaseControl.Visible := false; // Скрываем панель
    P_EditOperationInfo.Enabled := false; // Деактивируем панель
  end;
end;
```

Перед тем как прописывать код обработчика событий на редактирования, нам необходимо модифицировать обработчик события **AfterScroll** для набора данных **Q_v_operation_goods_list** следующим образом:

```
procedure TDataModule2.Q_v_operation_goods_listAfterScroll(DataSet:
  TDataSet);
begin
  // Проверяем, существует ли объект (создана ли форма Form5)
  if Assigned(Form5) then
  if Q_v_operation_goods_list.RecordCount > 0 then
  begin // Назначаем параметру идентификатор выбранного товара
    Q_v_selector_goods.Parameters[1].Value :=
      Q_v_operation_goods_listIDgc.Value;
    // Обновляем НД списка товаров для DBL_goods_selector
```



```

Q_v_selector_goods.Requery();
// Синхронизируем НД T_op_goods_list
T_op_goods_list.Locate('IDoperation;IDgc',
  VarArrayOf([Q_v_operation_goods_listIDoperation.Value,
    Q_v_operation_goods_listIDgc.Value]), []);
end;
end;

```

Вместо проверки создана ли форма **Form5** можно использовать механизм проверки полностью ли запустилось приложение или нет.

Для этого в блоке **var** основной формы (**Form1**) запишем объявление следующего вида: `IsAppStatrtedFlag:boolean = false;` и пропишем обработчик события **OnActivate** у формы:

```

procedure TForm1.FormActivate(Sender: TObject);
begin
  IsAppStatrtedFlag := true;
end;

```

Теперь условие `if Assigned(Form5) then` в обработчике события **AfterScroll** можно заменить на: `if IsAppStatrtedFlag then`. При этом следует помнить, что **DataModule2** должен быть связан с **Form1** (связь устанавливается при помощи пункта **Use Unit...** в меню **File**).

После настройки отображения можно прописать обработчики события **OnClick** для кнопок. Обработчик **OnClick** кнопки **Редактировать запись** (имя **B_ModifyGoods**):

```

procedure TForm5.B_ModifyGoodsClick(Sender: TObject);
begin
  if DataModule2.T_op_goods_list.State = dsBrowse then
  begin
    DBGrid1.Enabled := false; // Деактивируем компонент DBGrid1
    // Синхронизируем набор данных со списком выбора
    DataModule2.Q_v_selector_goods.Locate('IDgc',
      DataModule2.T_op_goods_listIDgc.Value, []);
    DataModule2.T_op_goods_list.edit; // Активируем редактирование НД
    P_ModifyData.Visible := true; // Показываем панель
    P_BaseControl.Visible := false; // Скрываем панель
    P_EditOperationInfo.Enabled := false; // Деактивируем панель
  end;
end;

```

Обработчик **OnClick** кнопки **Удалить запись** (**B_DeleteGoods**):

```

procedure TForm5.B_DeleteGoodsClick(Sender: TObject);
begin
  if DataModule2.T_op_goods_list.State = dsBrowse then
  if MessageDlg('Подтвердите удаление записи', mtConfirmation,
    [mbYes, mbNo], 0) = mrYes then
    DataModule2.T_op_goods_list.Delete;
end;

```

Обработчики **OnClick** для кнопок **Сохранить (B_SaveData)** и **Отмена (B_CancelModify)**:

```
procedure TForm5.B_SaveDataClick(Sender: TObject);
begin
  if DataModule2.T_op_goods_list.State in [dsEdit, dsInsert] then
  begin // Если находимся в режиме редактирования или добавления то
    DataModule2.T_op_goods_list.post; // Сохраняем запись НД
    P_BaseControl.Visible := true; // Показываем панель
    P_ModifyData.Visible := false; // Скрываем панель
    DBGrid1.Enabled := true; // Активируем компонент DBGrid1
    P_EditOperationInfo.Enabled := true; // Активируем панель
  end;
end;

procedure TForm5.B_CancelModifyClick(Sender: TObject);
begin
  if DataModule2.T_op_goods_list.State in [dsEdit, dsInsert] then
  begin // Если находимся в режиме редактирования или добавления то
    DataModule2.T_op_goods_list.Cancel; // Отменяем изменения в НД
    P_BaseControl.Visible := true; // Показываем панель
    P_ModifyData.Visible := false; // Скрываем панель
    DBGrid1.Enabled := true; // Активируем компонент DBGrid1
    P_EditOperationInfo.Enabled := true; // Активируем панель
  end;
end;
```

Теперь понадобится код, который будет обновлять данные на формах после их изменения в таблице **operations**, события **AfterPost** и **AfterDelete** :

```
procedure TDataModule2.T_operationsAfterPost(DataSet: TDataSet);
var
  IDoperation: integer;
begin
  IDoperation := T_operationsIDoperation.Value;
  Q_v_goods_operations.DisableControls();
  Q_v_goods_operations.Requery();
  Q_v_goods_operations.Locate('IDoperation', IDoperation, []);
  Q_v_goods_operations.EnableControls();
end;

procedure TDataModule2.T_operationsAfterDelete(DataSet: TDataSet);
begin
  Q_v_goods_operations.DisableControls();
  Q_v_goods_operations.Requery();
  Q_v_goods_operations.EnableControls();
end;
```

При изменении перечня товаров в составе операций нам также нужно обновить все наборы данных, которые используют данные таблицы

op_goods_list – **Q_v_operation_goods_list** и **Q_v_operations_fullcost**. Про-
пишите в обработчиках событий **AfterPost** и **AfterDelete** код:

```
procedure TDataModule2.T_op_goods_listAfterDelete(DataSet: TDataSet);  
begin  
    Q_v_operation_goods_list.DisableControls();  
    Q_v_operation_goods_list.Requery();  
    Q_v_operation_goods_list.EnableControls();  
    Q_v_operations_fullcost.Requery();  
end;
```

```
procedure TDataModule2.T_op_goods_listAfterPost(DataSet: TDataSet);  
var  
    IDgcVal: integer;  
begin  
    IDgcVal := T_op_goods_listIDgc.Value;  
    Q_v_operation_goods_list.DisableControls();  
    Q_v_operation_goods_list.Requery();  
    Q_v_operation_goods_list.Locate('IDgc', IDgcVal, []);  
    Q_v_operation_goods_list.EnableControls();  
    Q_v_operations_fullcost.Requery();  
end;
```

Поскольку реализован весь необходимый функционал для редакти-
рования параметров операции и ее состава, нам остается при закрытии
Form5 сделать проверку, что все данные сохранены.

При закрытии формы возникает событие **OnCloseQuery**, которое мы
будем использовать для обработки данных связанных с завершением
оформления операции. Будем проверять состав товаров в операции и если
количество записей о товарах будет равно нулю, то запись об операции
удаляем. Также если какие-либо данные небыли сохранены, то будет
предоставлена возможность сохранить их.

Обработчик события **OnCloseQuery** будет следующим:

```
procedure TForm5.FormCloseQuery(Sender: TObject;  
    var CanClose: Boolean);  
begin // Проверяем редактируем ли мы состав товаров в операции  
    if P_ModifyData.Visible then  
        begin  
            // Если мы редактируем данные, то выводим сообщение  
            ShowMessage ('В данный момент происходит изменения перечня товаров в  
рамках операции, вы не можете завершить оформление');  
            // Устанавливаем для флага CanClose значение говорящее,  
            // что нельзя закрыть форму  
            CanClose := false;  
            //при помощи команды exit() игнорируем оставшийся код  
            exit();  
        end;  
    // Если редактирование состава товаров операции не производится
```

```

// то проверяем количество товаров в операции и если их нет
if DataModule2.T_op_goods_list.RecordCount = 0 then
begin // то выполняем отмену изменений в таблице операций
  if DataModule2.T_operations.State = dsEdit then
    DataModule2.T_operations.Cancel;
  // Удаляем запись об операции
  DataModule2.T_operations.Delete;
end // В случае если в операции присутствуют товары,
// то проверяем есть ли изменения в данных
else if DataModule2.T_operations.State = dsEdit then
begin // В случае если требуется сохранить изменения,
  // запрашивается подтверждение сохранения
  if MessageDlg('Подтвердить сохранение измененных данных операции',
    mtConfirmation, [mbYes, mbNo], 0) = mrYes then
    // Сохраняем изменения при получении подтверждения
    DataModule2.T_operations.post
  else // в противном случае выполняем отмену
    DataModule2.T_operations.Cancel;
end;
// Обновляем данные в Q_v_goods_operations имитируя
// срабатывание события AfterPost
DataModule2.T_operationsAfterPost(nil);
// Обновляем данные в НД Q_v_storage
DataModule2.Q_v_storage.Requery;
end;

```

Настроив механизм изменения данных можно перейти к настройке вызова формы **Form5**.

3.5 Особенности вызова формы для редактирования данных с учетом перестройки интерфейса

Форма **Form5** отвечает за редактирование, как состава продаж, так и поставок, содержащихся в одной таблице. Ключевым отличием является указание поставщика при поставках. Таблица **operations** содержит поле **oIsSale** определяющее тип операции – **1** означает **Продажа**, **0** – **Поставка**.

Во время вызова формы возникает событие **OnShow**. Используя данное событие мы будем скрывать или же показывать компонент с именем **GB_Supplier** для выбора поставщика, а также изменять надпись компонента **GB_DateTime** для указания типа операции.

Код обработчика будет выглядеть следующим образом:

```

procedure TForm5.FormShow(Sender: TObject);
begin
  if DataModule2.T_operationsoIsSale.Value = 1 then
  begin
    GB_Supplier.Visible := false;
    GB_DateTime.Caption := 'Дата и время продажи товара';
  end
  else

```

```

begin
  GB_DateTime.Caption := 'Дата и время поставки товара';
  GB_Supplier.Visible := true;
end;
end;

```

Настройка **Form5** полностью завершена, теперь перейдем на главную форму (**Form1**) и реализуем механизм вызова **Form5**. Для этой цели мы будем использовать контекстное меню (при желании можно использовать кнопки).

Для создания контекстного меню разместите на **Form1** компонент **PopupMenu**. Настройка элементов в меню происходит аналогичным образом как в **MainMenu**. Создайте три элемента (рис. 3.20):

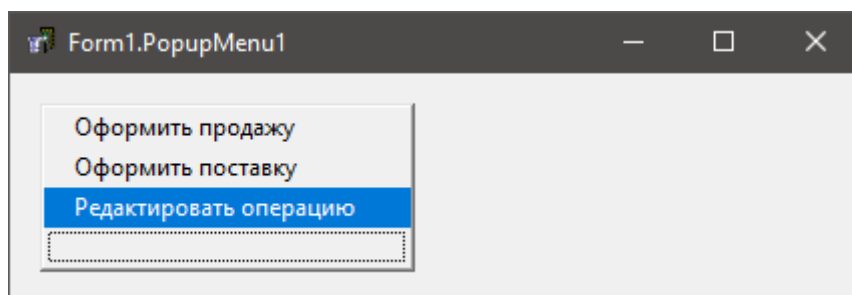


Рис. 3.20. Создание контекстного меню

Далее привяжем созданное меню к компоненту **DBGrid2** отображающему список операций. Для этого у **DBGrid2** в свойстве **PopupMenu** укажите имя созданного контекстного меню – **PopupMenu1**.

Поскольку все три элемента контекстного меню будут вызывать одну и ту же форму, то для сокращения объема кода создадим процедуру **DisplayForm5**. Данная процедура будет устанавливать дату и время из базы данных в компоненты **TDatePicker** и **TTimePicker**, а также вызывать необходимую форму (**Form5**).

Код процедуры **DisplayForm5**:

```

procedure DisplayForm5();
begin
  // Назначение даты (Trunc "отрезает" время)
  Form5.DatePicker1.Date :=
    Trunc(DataModule2.T_operationsoDateTime.value);
  // Назначение времени
  Form5.TimePicker1.Time :=
    TimeOf(DataModule2.T_operationsoDateTime.value);
  // Вызов показа формы
  Form5.ShowModal;
end;

```

Для работы функции **TimeOf** нужно прописать **dateutils** в блоке **uses** раздела **interface** модуля **Unit1**. Код процедуры следует разместить в начале функциональной части модуля раздела **implementation** (рис. 3.21):

```

unit Unit1;
interface
implementation
{$R *.dfm}

uses Unit2, Unit3, Unit4, Unit5;

procedure DisplayForm5();
begin
    // Назначение даты (Trunc "отрезает" время)
    Form5.DatePicker1.Date := Trunc(DataModule2.T_operationsoDateTime.value);
    // Назначение времени
    Form5.TimePicker1.Time := TimeOf(DataModule2.T_operationsoDateTime.value);
    // Вызов показа формы
    Form5.ShowModal;
end;

```

Рис. 3.21. Фрагмент кода модуля *Unit1*

Для событий **OnClick** элементов контекстного меню **Оформить продажу (PM_add_sale)**, **Оформить поставку (PM_add_supply)** и **Редактировать операцию (PM_Edit_Operation)** пропишем следующие обработчики:

```

procedure TForm1.PM_add_saleClick(Sender: TObject);
begin
    // функция Now() возвращает текущие дату и время
    DataModule2.T_operations.AppendRecord([nil, Now(), 1]);
    DisplayForm5();
end;

procedure TForm1.PM_add_supplyClick(Sender: TObject);
begin
    DataModule2.T_operations.AppendRecord([nil, Now(), 0]);
    DisplayForm5();
end;

procedure TForm1.PM_Edit_OperationClick(Sender: TObject);
begin
    DisplayForm5();
end;

```

Процедура **AppendRecord** заполняет поля таблицы значениями, которые передаются через массив **Values** (в квадратных скобках). При этом первое значение в массиве присваивается первому полю, второе – второму

и т. д. Естественно, что значения в массиве должны быть совместимы с полями таблицы. В случае если полю будет передано значение **nil**, оно сохранит свое текущее значение.

В нашем случае первое поле в наборе данных **T_operations – IDoperation** является автоинкрементным и соответственно значения для него генерируются в момент сохранения записи. Поэтому мы прописываем слово **nil**. Далее идет поле **oDateTime** с датой и временем, используя функцию **Now()** мы заносим текущее значения даты/времени. Третье поле **oIsSale** отвечает за тип операции, и мы в зависимости от элемента контекстного меню подставляем нужное значение.

Альтернативой для метода **AppendRecord** является метод **InsertRecord**, который отличается от него только тем, что помещает новую запись после текущей записи, а не вслед за последней записью набора данных.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Для чего применяется связь *Master–Detail*?
2. Что из себя представляет группировка данных (приведите пример сгруппированных данных)?
3. Опишите назначение агрегатных функций. Какие виды агрегирования поддерживает *MySQL*?
4. Каким образом в *MySQL* работает функция **IF**?
5. Для чего применяется функция **CONCAT**?
6. Дайте характеристику свойству **AlignWithMargins** визуальных компонентов.
7. Что из себя представляют параметры *SQL*-запросов (например, *:IDoperation*)?
8. Чем компонент **RadioGroup** отличается от **CheckListBox**?
9. Для каких целей применяется компонент **TGroupBox**?
10. Что описывает маска формата **hh:mm:ss**, какие еще варианты существуют?
11. Охарактеризуйте свойство **dgRowSelect**.
12. Для чего применяется процедура **Requery()**.
13. Чем **PopupMenu** отличается от **MainMenu**?
14. Для чего используется функция **Assigned**?
15. Для чего создаются уникальные индексы. Для чего предназначены индексы?
16. Что из себя представляет связь многие-ко-многим, приведите пример данных с такой связью?
17. Как реализуется связь многие-ко-многим в реляционной базе данных?
18. Дайте характеристику типу данных **Variant**, для чего он применяется?